

(text_embeddings) and unconditional embeddings (uncond_embeddings) into a single batch to avoid doing two forward passes.

```
text_embeddings = torch.cat([uncond_embeddings, text_embeddings])
```

Next, we can go ahead and generate the initial random noise.

```
latents = torch.randn(
    (batch_size, unet.in_channels, height // 8, width // 8),
    generator=generator,
)
```

```
latents = latents.to(torch_device)
```

The shape of the latents at this point is torch.Size([1, 4, 64, 64]), which is much smaller than the desired image size of 512 x 512. The model will use this latent representation, which contains pure noise, to generate a 512 x 512 image in the next step.

Then, we initialize the scheduler with the selected num_inference_steps. The scheduler will compute the sigmas and time step values that will be used during the denoising process.

```
scheduler.set_timesteps(num_inference_steps)
```

The K-LMS scheduler needs to multiply the latents by its sigma values.

Let's do this here:

```
latents = latents * scheduler.init_noise_sigma
```

We are ready to write the denoising loop.

```
from tqdm.auto import tqdm
```

```
scheduler.set_timesteps(num_inference_steps)
```

```
for t in tqdm(scheduler.timesteps):
```

expand the latents if we are doing classifier-free guidance to avoid doing two forward passes.

```
latent_model_input = torch.cat([latents] * 2)
```

```
latent_model_input = scheduler.scale_model_input(latent_model_input, timestep=t)
```

```
# predict the noise residual
```

```
with torch.no_grad():
```

```
noise_pred = unet(latent_model_input, t, encoder_hidden_states=text_embeddings).sample
```

```
# perform guidance
```

```
noise_pred_uncond, noise_pred_text = noise_pred.chunk(2)
```

```
noise_pred = noise_pred_uncond + guidance_scale * (noise_pred_text - noise_pred_uncond)
```

```
# compute the previous noisy sample x_t -> x_t-1
```